Cloud-Native CI/CD Pipelines with GitOps

Next.js + NestJS real-time game platform deployed with GitHub Actions, ArgoCD, and AWS EKS

Repository: https://github.com/AdelHussein01/CloudProj

Executive Summary

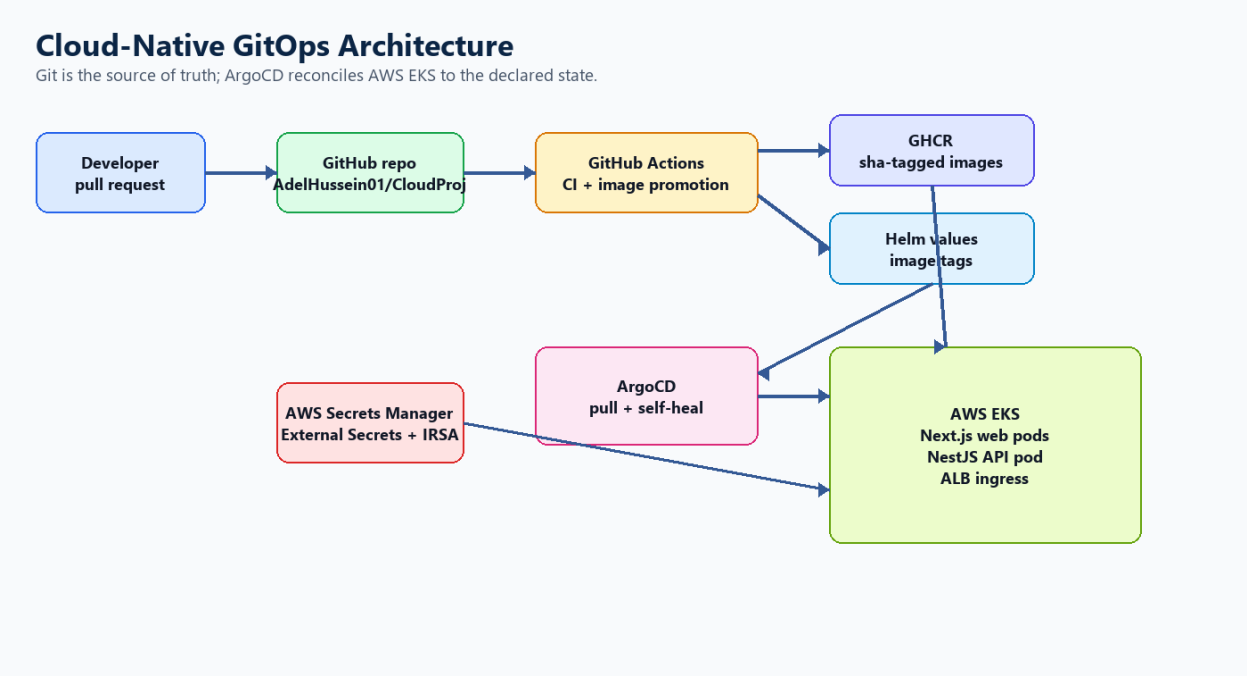
This project implements a GitOps-based CI/CD pipeline for a cloud-native two-player game platform. The application lets a host create a private link, choose XO or rock-paper-scissors, and invite the first user who opens the link to join the selected game in real time.

The technical goal is to show how declarative infrastructure, immutable image promotion, automated testing, secure secret management, rollback, and observability combine into a reliable delivery workflow.

Project Scope

- Application: Next.js frontend and NestJS Socket.IO API.
- CI: GitHub Actions for type checks, tests, builds, Docker image publishing, and image scanning.
- CD: ArgoCD watches Helm manifests and reconciles AWS EKS to the Git-declared desired state.
- Secrets: AWS Secrets Manager, External Secrets Operator, and IAM Roles for Service Accounts.
- Rollback: Git revert or a controlled rollback workflow that commits known-good image tags.
- AWS execution is intentionally left as the final phase to protect account access and control cost.

System Architecture



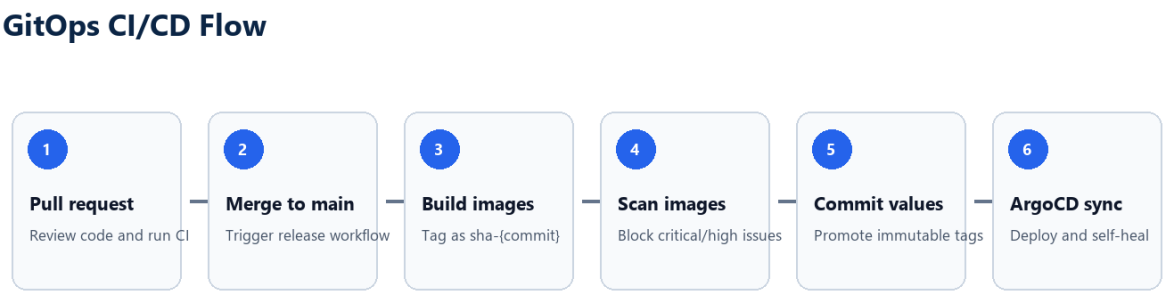
The architecture separates CI from CD. GitHub Actions validates code and promotes immutable image tags to Git, while ArgoCD performs cluster reconciliation from inside Kubernetes.

Application Design

Layer	Technology	Responsibility
Web	Next.js	Room creation, share links, join screen, XO board, RPS rounds, responsive UI.
API	NestJS + Socket.IO	Real-time rooms, player state, game validation, broadcasts, health endpoint.
Packaging	Docker	Separate production images for web and API.
Runtime	Kubernetes	Deployments, services, ingress, probes, resources, and optional autoscaling.

The API stores room state in memory and therefore starts with one API replica for the demo. A production version should add Redis or another shared state backend before horizontal API scaling.

CI/CD Pipeline



1. A developer opens a pull request.
2. GitHub Actions runs type checks, tests, and production builds.
3. After merge to main, Docker images are built and pushed to GitHub Container Registry.
4. Images are tagged with the commit SHA to preserve immutability.
5. Trivy scans the images and blocks high or critical vulnerabilities.
6. The release workflow commits updated Helm image tags.
7. ArgoCD detects the Git change and syncs AWS EKS.

GitOps vs Traditional CI/CD

Area	Traditional CI/CD	GitOps in This Project
Deployment actor	Pipeline pushes to the cluster.	ArgoCD pulls desired state from Git.
Source of truth	Pipeline state and live cluster can diverge.	Git stores the declared runtime state.
Rollback	Re-run a pipeline or manually deploy an older artifact.	Revert Git or commit previous immutable tags.
Drift handling	Manual checks or periodic audits.	ArgoCD detects and self-heals drift.
Audit trail	Pipeline logs plus release notes.	Git history plus ArgoCD sync history.

Security and Secret Management

- No raw secrets are committed to Git.
- GitHub Actions uses scoped permissions for CI, package publishing, and rollback.
- Runtime secrets are stored in AWS Secrets Manager and synced by External Secrets Operator.
- External Secrets authenticates through IRSA, avoiding static AWS keys in pods.
- Image tags use commit SHAs, not mutable production tags.
- Container images are scanned before promotion.

Rollback Strategy

The preferred rollback is a Git operation. Reverting the GitOps promotion commit or running the manual rollback workflow updates Helm values with known-good image tags. ArgoCD then reconciles the cluster to that older declared state.

- Fast rollback path: run the rollback workflow with previous sha-* image tags.
- Audit-friendly rollback path: revert the bad promotion commit.
- Avoid cluster-only rollback because it creates drift between Git and Kubernetes.

Observability and Evaluation

Metric	Purpose	How to Measure
Deployment frequency	Shows release throughput.	Count successful release workflow runs.
Lead time	Measures merge-to-production speed.	Time from merge to healthy ArgoCD sync.
Change failure rate	Measures deployment quality.	Failed deployments divided by total deployments.
MTTR	Measures recovery speed.	Time from bad release detection to healthy rollback.
Drift repair time	Measures GitOps self-healing.	Time from manual cluster drift to ArgoCD correction.

Demo Plan

1. Open the deployed application.
2. Create an XO room and copy the share link.
3. Join from a second browser and finish an XO match.
4. Create a rock-paper-scissors room and play multiple rounds.
5. Open GitHub Actions and show CI and release workflows.
6. Open ArgoCD and show sync status and application health.
7. Create controlled drift by scaling the web deployment and show ArgoCD self-healing.
8. Run the rollback workflow or revert a promotion commit and verify the older version is restored.

AWS Final Phase

AWS deployment should be performed last. This avoids exposing credentials early, gives time to review the project artifacts, and allows the team to confirm cost and cleanup expectations before creating paid cloud resources.

- Create EKS infrastructure with Terraform.
- Install AWS Load Balancer Controller, External Secrets Operator, and ArgoCD.
- Create the required AWS Secrets Manager entry.
- Apply ArgoCD manifests and verify the application sync.
- Use a real domain or the AWS ALB DNS name for the demo.

Conclusion

GitOps improves reliability by making Git the system of record for deployment state. In this project, every deployment and rollback is traceable, automated, and reproducible. The main engineering risks are cloud IAM setup, secret handling, and state sharing for real-time WebSocket workloads.

References

- Amazon EKS User Guide: https://docs.aws.amazon.com/eks/latest/userguide/
- AWS Load Balancer Controller on EKS: https://docs.aws.amazon.com/eks/latest/userguide/lbc-helm.html
- ArgoCD documentation: https://argo-cd.readthedocs.io/
- GitHub Actions documentation: https://docs.github.com/en/actions
- GitHub Container Registry documentation: https://docs.github.com/en/packages/working-with-a-github-packages-registry/working-with-the-container-registry
- External Secrets Operator documentation: https://external-secrets.io/